유틸리티 시스템 분석

1. 유틸리티 시스템 개요

Open R1의 유틸리티 시스템은 훈련, 평가, 데이터 처리, 모델 관리 등 다양한 기능을 지원하는 헬퍼 함수들과 클래스들로 구성되어 있습니다. 이 시스템은 코드의 재사용성을 높이고 복잡한 작업을 단순화합니다.

1.1 주요 유틸리티 모듈

- **data.py**: 데이터셋 로딩 및 믹스처 생성
- **model_utils.py**: 모델 및 토크나이저 로딩
- **callbacks.py**: 훈련 콜백 시스템
- **hub.py**: HuggingFace Hub 통합
- **wandb_logging.py**: Weights & Biases 로깅
- **import_utils.py**: 의존성 확인 유틸리티
- **evaluation.py**: 평가 시스템 통합

1.2 시스템 아키텍처

```
유틸리티 시스템
├── 데이터 처리 (data.py)
├── 모델 관리 (model_utils.py)
├── 훈련 지원 (callbacks.py)
├── 클라우드 통합 (hub.py, wandb_logging.py)
```

```
├── 의존성 관리 (import_utils.py)
└── 평가 지원 (evaluation.py)
```

2. 데이터 처리 유틸리티 (`src/open_r1/utils/data.py`)

2.1 데이터셋 로딩 시스템

```
def  get_dataset(args: ScriptArguments) -> DatasetDict:
"""설정에 따라 데이터셋 또는 데이터셋 믹스처 로드
Args:
args: 데이터셋 설정이 포함된 스크립트 인수
Returns:
DatasetDict: 로드된 데이터셋
"""
# 단일 데이터셋 로딩
if args.dataset_name and not args.dataset_mixture:
logger.info(f"Loading dataset: {args.dataset_name}")
return datasets.load_dataset(args.dataset_name, args.dataset_config)
# 데이터셋 믹스처 생성
elif args.dataset_mixture:
logger.info(f"Creating dataset mixture with {len(args.dataset_mixture.datasets)}
return create_dataset_mixture(args.dataset_mixture)
else:
```

```
raise ValueError("Either `dataset_name` or `dataset_mixture` must be provided")
```

2.2 데이터셋 믹스처 생성

```
def  create_dataset_mixture(mixture_config) -> DatasetDict:

"""여러 데이터셋을 조합하여 믹스처 생성"""

seed = mixture_config.seed

datasets_list = []

# 각 데이터셋 로드 및 전처리

for dataset_config in mixture_config.datasets:

logger.info(f"Loading dataset for mixture: {dataset_config.id}")

# 데이터셋 로드

ds = datasets.load_dataset(

dataset_config.id,

dataset_config.config,

split=dataset_config.split,

)

# 컬럼 선택 (필요한 경우)

if dataset_config.columns is not None:

ds = ds.select_columns(dataset_config.columns)

# 가중치 적용 (서브샘플링)

if dataset_config.weight is not None:
```

```
sample_size = int(len(ds) * dataset_config.weight)

ds = ds.shuffle(seed=seed).select(range(sample_size))

logger.info(

f"Subsampled dataset '{dataset_config.id}' "

f"with weight={dataset_config.weight} to {len(ds)} examples"

)

datasets_list.append(ds)

# 데이터셋 결합 및 셔플

if datasets_list:

combined_dataset = concatenate_datasets(datasets_list)

combined_dataset = combined_dataset.shuffle(seed=seed)

logger.info(f"Created dataset mixture with {len(combined_dataset)} examples")

# 테스트 분할 (선택적)

if mixture_config.test_split_size is not None:

combined_dataset = combined_dataset.train_test_split(

test_size=mixture_config.test_split_size,

seed=seed

)

logger.info(

f"Split dataset into train and test sets "

f"with test size: {mixture_config.test_split_size}"

)

return combined_dataset
```

```
else:
return DatasetDict({"train": combined_dataset})
else:
raise ValueError("No datasets were loaded from the mixture configuration")
```

2.3 데이터셋 믹스처 설정 예시

```
# YAML 설정 예시
dataset_mixture:
datasets:
# 수학 문제 (60% 가중치)
- id: open-r1/OpenR1-Math-220k
config: default
split: train
columns: ["prompt", "response", "solution"]
weight: 0.6
# 코딩 문제 (30% 가중치)
- id: open-r1/codeforces-cots
config: default
split: train
columns: ["prompt", "response"]
weight: 0.3
# 과학 문제 (10% 가중치)
```

```yaml
- id: allenai/ai2_arc
  config: ARC-Challenge
  split: train
  columns: ["question", "choices", "answerKey"]
  weight: 0.1
  seed: 42
  test_split_size: 0.1
```

3. 모델 관리 유틸리티 (`src/open_r1/utils/model_utils.py`)

3.1 토크나이저 로딩

```python
def get_tokenizer(
model_args: ModelConfig,
training_args: SFTConfig | GRPOConfig
) -> PreTrainedTokenizer:
"""모델용 토크나이저 로드"""
# 기본 토크나이저 로드
tokenizer = AutoTokenizer.from_pretrained(
model_args.model_name_or_path,
revision=model_args.model_revision,
trust_remote_code=model_args.trust_remote_code,
)
```

```
# 커스텀 채팅 템플릿 적용 (선택적)
if training_args.chat_template is not None:
tokenizer.chat_template = training_args.chat_template
return tokenizer
```

3.2 모델 로딩

```
def  get_model(
model_args: ModelConfig,
training_args: SFTConfig | GRPOConfig
) -> AutoModelForCausalLM:
"""모델 로드 및 설정"""
# 데이터 타입 설정
torch_dtype = (
model_args.torch_dtype
if model_args.torch_dtype in ["auto", None]
else  getattr(torch, model_args.torch_dtype)
)
# 양자화 설정
quantization_config = get_quantization_config(model_args)
# 모델 로딩 매개변수
model_kwargs = dict(
revision=model_args.model_revision,
```

```
trust_remote_code=model_args.trust_remote_code,
attn_implementation=model_args.attn_implementation,
torch_dtype=torch_dtype,
use_cache=False if training_args.gradient_checkpointing else True,
device_map=get_kbit_device_map() if quantization_config is  not None else None,
quantization_config=quantization_config,
)
# 모델 로드
model = AutoModelForCausalLM.from_pretrained(
model_args.model_name_or_path,
**model_kwargs,
)
return model
```

3.3 모델 설정 최적화

```
def  optimize_model_config(model_args: ModelConfig, training_args) -> dict:
"""하드웨어 및 훈련 설정에 따른 모델 최적화"""
config = {}
# 어텐션 구현 최적화
if  hasattr(training_args, 'use_flash_attention') and training_args.use_flash_at
config['attn_implementation'] = 'flash_attention_2'
# 그래디언트 체크포인팅 설정
```

```
if training_args.gradient_checkpointing:

config['use_cache'] = False

config['gradient_checkpointing'] = True

# 메모리 효율적 어텐션

if  hasattr(training_args, 'use_memory_efficient_attention'):

config['use_memory_efficient_attention'] = training_args.use_memory_efficient_at

# 양자화 설정

if model_args.load_in_8bit or model_args.load_in_4bit:

config['quantization_config'] = get_quantization_config(model_args)

config['device_map'] = get_kbit_device_map()

return config
```

4. 훈련 콜백 시스템 (src/open_r1/utils/callbacks.py)

4.1 Hub 푸시 콜백

```
class PushToHubRevisionCallback(TrainerCallback):

"""훈련 중 체크포인트를 HuggingFace Hub에 자동 푸시"""

def  __init__(self, model_config) -> None:

self.model_config = model_config


def  on_save(

self,
```

```
args: TrainingArguments,
state: TrainerState,
control: TrainerControl,
**kwargs,
):
"""체크포인트 저장 시 Hub에 푸시"""
if state.is_world_process_zero:
global_step = state.global_step
# 체크포인트별 브랜치 생성
dummy_config = DummyConfig(
hub_model_id=args.hub_model_id,
hub_model_revision=f"{args.hub_model_revision}-step-{global_step:09d}",
output_dir=f"{args.output_dir}/checkpoint-{global_step}",
system_prompt=args.system_prompt,
)
# Hub에 비동기 푸시
future = push_to_hub_revision(
dummy_config,
extra_ignore_patterns=["*.pt"] # 옵티마이저 상태 제외
)
# Slurm 환경에서 벤치마크 자동 실행
if is_slurm_available():
dummy_config.benchmarks = args.benchmarks
```

```
def  run_benchmark_callback(_):
print(f"Checkpoint {global_step} pushed to hub.")
run_benchmark_jobs(dummy_config, self.model_config)
future.add_done_callback(run_benchmark_callback)
```

4.2 커스텀 콜백 등록

```
# 사용 가능한 콜백들
CALLBACKS = {
"push_to_hub_revision": PushToHubRevisionCallback,
# 추가 콜백들을 여기에 등록
}

def  get_callbacks(train_config, model_config) -> List[TrainerCallback]:
"""설정에 따른 콜백 목록 생성"""
callbacks = []
for callback_name in train_config.callbacks:
if callback_name not in CALLBACKS:
raise ValueError(f"Callback {callback_name} not found in CALLBACKS.")
callbacks.append(CALLBACKS[callback_name](model_config))
return callbacks
```

4.3 추가 콜백 예시

```python
class WandbLoggingCallback(TrainerCallback):
"""Weights & Biases 로깅 콜백"""
def __init__(self, project_name: str = None):
self.project_name = project_name
def on_train_begin(self, args, state, control, **kwargs):
"""훈련 시작 시 W&B 초기화"""
import wandb
wandb.init(
project=self.project_name or "open-r1",
config=args.to_dict(),
name=f"{args.run_name}-{state.global_step}"
)
def on_log(self, args, state, control, logs=None, **kwargs):
"""로그 발생 시 W&B에 전송"""
if logs:
import wandb
wandb.log(logs, step=state.global_step)


class EarlyStoppingCallback(TrainerCallback):
"""조기 종료 콜백"""
def __init__(self, patience: int = 3, min_delta: float = 0.001):
self.patience = patience
```

```
self.min_delta = min_delta

self.best_metric = None

self.patience_counter = 0

def  on_evaluate(self, args, state, control, logs=None, **kwargs):

"""평가 후 조기 종료 조건 확인"""

if logs and "eval_loss" in logs:

current_metric = logs["eval_loss"]

if  self.best_metric is None or current_metric < self.best_metric - self.min_del

self.best_metric = current_metric

self.patience_counter = 0

else:

self.patience_counter += 1

if  self.patience_counter >= self.patience:

print(f"Early stopping triggered after {self.patience} evaluations without impro

control.should_training_stop = True
```

5. HuggingFace Hub 통합 (`src/open_r1/utils/hub.py`)

5.1 Hub 푸시 기능

```
def  push_to_hub_revision(

training_args: SFTConfig | GRPOConfig,

extra_ignore_patterns=[]
```

```
) -> Future:
"""모델을 Hub 리포지토리의 브랜치에 푸시"""
# 리포지토리 생성 (존재하지 않는 경우)
repo_url = create_repo(
repo_id=training_args.hub_model_id,
private=True,
exist_ok=True
)
# 브랜치 생성을 위한 초기 커밋 가져오기
initial_commit = list_repo_commits(training_args.hub_model_id)[-1]
# 타겟 브랜치 생성
create_branch(
repo_id=training_args.hub_model_id,
branch=training_args.hub_model_revision,
revision=initial_commit.commit_id,
exist_ok=True,
)
logger.info(f"Created target repo at {repo_url}")
logger.info(f"Pushing to the Hub revision {training_args.hub_model_revision}..."
# 무시할 파일 패턴 설정
ignore_patterns = ["checkpoint-*", "*.pth"]
ignore_patterns.extend(extra_ignore_patterns)
# 비동기 업로드
```

```
future = upload_folder(
repo_id=training_args.hub_model_id,
folder_path=training_args.output_dir,
revision=training_args.hub_model_revision,
commit_message=f"Add {training_args.hub_model_revision} checkpoint",
ignore_patterns=ignore_patterns,
run_as_future=True,
)
logger.info(f"Pushed to {repo_url} revision {training_args.hub_model_revision} s
return future
```

5.2 Hub 리비전 확인

```
def  check_hub_revision_exists(training_args: SFTConfig | GRPOConfig):
"""주어진 Hub 리비전이 존재하는지 확인"""
if repo_exists(training_args.hub_model_id):
if training_args.push_to_hub_revision is True:
# 리비전 존재 여부 확인
revisions = [rev.name for rev in list_repo_refs(training_args.hub_model_id).bran
if training_args.hub_model_revision in revisions:
# README 파일 존재 여부 확인
repo_files = list_repo_files(
repo_id=training_args.hub_model_id,
```

```
revision=training_args.hub_model_revision,
)
if "README.md" in repo_files and training_args.overwrite_hub_revision is False:
raise ValueError(
f"Revision {training_args.hub_model_revision} already exists. "
"Use --overwrite_hub_revision to overwrite it."
)
```

5.3 모델 매개변수 수 추정

```
def get_param_count_from_repo_id(repo_id: str) -> int:
"""리포지토리 ID에서 모델 매개변수 수 추정"""
try:
# safetensors 메타데이터에서 정확한 매개변수 수 가져오기
metadata = get_safetensors_metadata(repo_id)
return list(metadata.parameter_count.values())[0]
except Exception:
# 리포지토리 이름에서 패턴 매칭으로 추정
# 예: "42m", "1.5b", "8x7b" 등의 패턴 인식
pattern = r"((\d+(\.\d+)?)(x(\d+(\.\d+)?))?)([bm])"
matches = re.findall(pattern, repo_id.lower())
param_counts = []
for full_match, number1, _, _, number2, _, unit in matches:
```

```
# 첫 번째 숫자 파싱
num1 = float(number1)
# 곱셈 패턴 처리 (예: 8x7b)
if number2:
num2 = float(number2)
total = num1 * num2
else:
total = num1
# 단위 변환 (b=billion, m=million)
if unit == 'b':
param_counts.append(int(total * 1_000_000_000))
elif unit == 'm':
param_counts.append(int(total * 1_000_000))
# 가장 큰 값 반환 (일반적으로 모델 크기)
return max(param_counts) if param_counts else 0
```

5.4 GPU 수 자동 결정

```
def get_gpu_count_for_vllm(model_name: str, model_revision: str = "main") -> in
"""vLLM 실행에 필요한 GPU 수 자동 결정"""
param_count = get_param_count_from_repo_id(model_name)
# 매개변수 수에 따른 GPU 수 결정
if param_count >= 70_000_000_000: # 70B+
```

```
return 8
elif param_count >= 30_000_000_000: # 30B-70B
return 4
elif param_count >= 13_000_000_000: # 13B-30B
return 2
else: # 13B 미만
return 1
```

6. Weights & Biases 통합 (`src/open_r1/utils/wandb_logging.py`)

6.1 W&B 초기화

```
def init_wandb_training(training_args):
"""Weights & Biases 로깅 도구 설정"""
# 환경 변수 설정
if training_args.wandb_entity is not None:
os.environ["WANDB_ENTITY"] = training_args.wandb_entity
if training_args.wandb_project is not None:
os.environ["WANDB_PROJECT"] = training_args.wandb_project
if training_args.wandb_run_group is not None:
os.environ["WANDB_RUN_GROUP"] = training_args.wandb_run_group
```

6.2 고급 W&B 로깅

```
def setup_advanced_wandb_logging(training_args, model_config):
"""고급 W&B 로깅 설정"""
import wandb
# 실행 초기화
wandb.init(
project=training_args.wandb_project,
entity=training_args.wandb_entity,
group=training_args.wandb_run_group,
name=f"{training_args.run_name}-{training_args.hub_model_revision}",
config={
**training_args.to_dict(),
**model_config.to_dict(),
},
tags=["open-r1", training_args.task_type],
)
# 모델 아키텍처 로깅
if hasattr(model_config, 'model_name_or_path'):
wandb.config.update({
"model_name": model_config.model_name_or_path,
"model_params": get_param_count_from_repo_id(model_config.model_name_or_path),
})
```

```
def log_training_metrics(metrics: dict, step: int):
"""훈련 메트릭 로깅"""
import wandb
# 기본 메트릭 로깅
wandb.log(metrics, step=step)
# 추가 계산된 메트릭
if "train_loss" in metrics and "eval_loss" in metrics:
wandb.log({
"loss_ratio": metrics["eval_loss"] / metrics["train_loss"],
"overfitting_indicator": max(0, metrics["eval_loss"] - metrics["train_loss"])
}, step=step)


def log_generation_samples(samples: list, step: int, max_samples: int = 5):
"""생성 샘플 로깅"""
import wandb
# 테이블 형태로 샘플 로깅
table = wandb.Table(columns=["prompt", "generated", "step"])
for i, sample in enumerate(samples[:max_samples]):
table.add_data(
sample.get("prompt", ""),
sample.get("generated", ""),
step
```

```
)

wandb.log({"generation_samples": table}, step=step)
```

7. 의존성 관리 (`src/open_r1/utils/import_utils.py`)

7.1 패키지 가용성 확인

```
from transformers.utils.import_utils import _is_package_available


# E2B 가용성 확인
_e2b_available = _is_package_available("e2b")


def  is_e2b_available() -> bool:
"""E2B 패키지 가용성 확인"""
return _e2b_available


# MorphCloud 가용성 확인
_morph_available = _is_package_available("morphcloud")


def  is_morph_available() -> bool:
"""MorphCloud 패키지 가용성 확인"""
return _morph_available
```

7.2 조건부 임포트 패턴

```
# 안전한 조건부 임포트 예시
def safe_import_with_fallback():
"""의존성이 없는 경우 대체 구현 제공"""
try:
from e2b_code_interpreter import AsyncSandbox
return AsyncSandbox
except ImportError:
# 대체 구현 또는 더미 클래스
class DummySandbox:
def __init__(self):
raise ImportError(
"E2B is not available. Please install with "
"`pip install e2b-code-interpreter`"
)
return DummySandbox

# 런타임 의존성 확인
def check_runtime_dependencies():
"""런타임에 필요한 의존성 확인"""
missing_deps = []
if not is_e2b_available():
```

```
missing_deps.append("e2b-code-interpreter")

if not is_morph_available():

missing_deps.append("morphcloud")

if missing_deps:

raise ImportError(

f"Missing required dependencies: {', '.join(missing_deps)}\n"

f"Please install them with: pip install {' '.join(missing_deps)}"

)
```

7.3 기능별 의존성 그룹

```
# 기능별 의존성 정의

FEATURE_DEPENDENCIES = {

"code_execution": ["e2b-code-interpreter", "morphcloud"],

"evaluation": ["lighteval", "math-verify"],

"data_generation": ["distilabel", "vllm"],

"logging": ["wandb", "tensorboard"],

"hub_integration": ["huggingface-hub"],

}


def  check_feature_dependencies(feature: str) -> bool:

"""특정 기능에 필요한 의존성 확인"""

if feature not in FEATURE_DEPENDENCIES:
```

```
raise ValueError(f"Unknown feature: {feature}")

deps = FEATURE_DEPENDENCIES[feature]

missing = []

for dep in deps:

if not _is_package_available(dep.replace("-", "_")):

missing.append(dep)

if missing:

print(f"Warning: Missing dependencies for {feature}: {missing}")

return False

return True



def  get_available_features() -> list:

"""사용 가능한 기능 목록 반환"""

available = []

for feature in FEATURE_DEPENDENCIES:

if check_feature_dependencies(feature):

available.append(feature)

return available
```

8. 설정 관리 유틸리티

8.1 설정 검증

```
def  validate_training_config(training_args, model_args):
"""훈련 설정 검증"""
errors = []
warnings = []
# 기본 검증
if training_args.learning_rate <= 0:
errors.append("Learning rate must be positive")
if training_args.per_device_train_batch_size <= 0:
errors.append("Batch size must be positive")
# 메모리 관련 검증
if training_args.gradient_checkpointing and training_args.dataloader_pin_memory:
warnings.append("Gradient checkpointing with pin_memory may cause memory issues"
# 모델 특화 검증
param_count = get_param_count_from_repo_id(model_args.model_name_or_path)
if param_count > 30_000_000_000 and training_args.per_device_train_batch_size >
warnings.append(f"Large model ({param_count/1e9:.1f}B params) with batch size >
# Hub 설정 검증
if training_args.push_to_hub and not training_args.hub_model_id:
errors.append("hub_model_id is required when push_to_hub is True")
# 오류 및 경고 출력
if errors:
raise ValueError(f"Configuration errors:\n" + "\n".join(f"- {e}"  for e in error
```

```
if warnings:
print("Configuration warnings:")
for w in warnings:
print(f"- {w}")


def  optimize_config_for_hardware(training_args, available_gpus: int, gpu_memory
"""하드웨어에 맞춘 설정 최적화"""
# GPU 수에 따른 배치 크기 조정
if available_gpus > 1:
# 멀티 GPU 환경에서는 배치 크기를 GPU 수에 비례하여 증가
training_args.per_device_train_batch_size = max(
1, training_args.per_device_train_batch_size // available_gpus
)
# GPU 메모리에 따른 설정 조정
if gpu_memory_gb < 24:
# 메모리가 부족한 경우
training_args.gradient_checkpointing = True
training_args.dataloader_pin_memory = False
training_args.per_device_train_batch_size = min(
training_args.per_device_train_batch_size, 2
)
elif gpu_memory_gb >= 80:
# 메모리가 충분한 경우
```

```
training_args.per_device_train_batch_size = min(
training_args.per_device_train_batch_size * 2, 16
)
return training_args
```

8.2 환경 감지

```
def  detect_environment():
"""실행 환경 감지"""
env_info = {
"platform": platform.system(),
"python_version": platform.python_version(),
"cuda_available": torch.cuda.is_available(),
"cuda_version": torch.version.cuda if torch.cuda.is_available() else None,
"gpu_count": torch.cuda.device_count() if torch.cuda.is_available() else  0,
"slurm_available": is_slurm_available(),
"distributed": int(os.environ.get("WORLD_SIZE", 1)) > 1,
}
# GPU 메모리 정보
if env_info["cuda_available"]:
gpu_memory = []
for i in  range(env_info["gpu_count"]):
memory_gb = torch.cuda.get_device_properties(i).total_memory / 1e9
```

```
gpu_memory.append(memory_gb)

env_info["gpu_memory_gb"] = gpu_memory

# 클러스터 환경 감지

if  "SLURM_JOB_ID" in os.environ:

env_info["slurm_job_id"] = os.environ["SLURM_JOB_ID"]

env_info["slurm_node_count"] = int(os.environ.get("SLURM_NNODES", 1))

env_info["slurm_tasks_per_node"] = int(os.environ.get("SLURM_NTASKS_PER_NODE", 1

return env_info



def  print_environment_info():

"""환경 정보 출력"""

env_info = detect_environment()

print("=== Environment Information ===")

print(f"Platform: {env_info['platform']}")

print(f"Python: {env_info['python_version']}")

print(f"CUDA Available: {env_info['cuda_available']}")

if env_info["cuda_available"]:

print(f"CUDA Version: {env_info['cuda_version']}")

print(f"GPU Count: {env_info['gpu_count']}")

for i, memory in  enumerate(env_info.get("gpu_memory_gb", [])):

print(f" GPU {i}: {memory:.1f} GB")

print(f"Slurm Available: {env_info['slurm_available']}")

print(f"Distributed: {env_info['distributed']}")
```

```
if "slurm_job_id" in env_info:
print(f"Slurm Job ID: {env_info['slurm_job_id']}")
print(f"Nodes: {env_info['slurm_node_count']}")
print(f"Tasks per Node: {env_info['slurm_tasks_per_node']}")
print("=" * 32)
```

9. 실제 사용 예시

9.1 완전한 훈련 설정

```
from open_r1.utils import (
get_dataset, get_tokenizer, get_model, get_callbacks,
init_wandb_training, validate_training_config,
detect_environment, print_environment_info
)

def setup_training():
"""완전한 훈련 환경 설정"""
# 환경 정보 출력
print_environment_info()
# 설정 로드
training_args = SFTConfig(...)
model_args = ModelConfig(...)
```

```
script_args = ScriptArguments(...)

# 설정 검증

validate_training_config(training_args, model_args)

# W&B 초기화

if  "wandb" in training_args.report_to:

init_wandb_training(training_args)

# 데이터셋 로드

dataset = get_dataset(script_args)

# 모델 및 토크나이저 로드

tokenizer = get_tokenizer(model_args, training_args)

model = get_model(model_args, training_args)

# 콜백 설정

callbacks = get_callbacks(training_args, model_args)

return {

"dataset": dataset,

"tokenizer": tokenizer,

"model": model,

"callbacks": callbacks,

}
```

9.2 자동화된 실험 관리

```
def  run_automated_experiment(config_path: str):
```

```
"""자동화된 실험 실행"""

# 설정 로드

with  open(config_path) as f:

config = yaml.safe_load(f)

# 환경 감지 및 최적화

env_info = detect_environment()

config = optimize_config_for_hardware(

config,

env_info["gpu_count"],

max(env_info.get("gpu_memory_gb", [0]))

)

# 의존성 확인

required_features = config.get("required_features", [])

for feature in required_features:

if not check_feature_dependencies(feature):

raise RuntimeError(f"Required feature '{feature}' dependencies not available")

# 훈련 실행

setup = setup_training()

# 결과 저장 및 정리

try:

trainer = SFTTrainer(**setup)

trainer.train()

# 자동 평가
```

```
if config.get("auto_evaluate", False):

run_benchmark_jobs(training_args, model_args)

# 자동 Hub 푸시

if config.get("auto_push", False):

push_to_hub_revision(training_args)

except Exception as e:

logger.error(f"Training failed: {e}")

raise

finally:

# 정리 작업

cleanup_resources()
```

유틸리티 시스템은 Open R1의 모든 구성 요소를 지원하는 기반 인프라로, 코드의 재사용성과 유지보수성을 크게 향상시킵니다. 각 유틸리티는 특정 기능에 특화되어 있으면서도 서로 유기적으로 연결되어 전체 시스템의 효율성을 높입니다.